

PLAN DE DESARROLLO DEL SOFTWARE



Constructing

P02

Versión: Final

Fecha del documento: 07 de Junio de 2026

Fecha de Inicio del Proyecto: Marzo 2026

Fecha fin del Proyecto: Diciembre 2026

del Río Nicolás Sebastián



Este documento está basado en el Software Delopment Plan (SDP) creado por Construx Software Builders, el cual es una adaptación de *Software Project Survival Guide* de Steve McConnell

TABLA DE REVISIÓN

Versión	Autor (es) Primario (s)	Descripción de la Versión	Fecha de Finalización
Borrador	del Río Nicolás	Borrador inicial creado para la distribución y la revisión de comentarios.	24/05/2026
Preliminar	del Río Nicolás	Segundo borrador con incorporación del examen inicial de los comentarios, distribuido para una revisión final	28/05/2026
Final	del Río Nicolás	Primer borrador completo, que se encuentra bajo el control de cambios.	07/06/2026

PREFACIO

El presente Plan de Desarrollo de Software (SDP) define las directrices, procesos técnicos y metodologías de gestión que guiarán la construcción del ecosistema **ConstructING**. Su propósito principal es servir como documento rector para la planificación, ejecución y control del proyecto, asegurando que la solución tecnológica se desarrolle de manera estructurada, predecible y alineada con los objetivos del negocio.

Basado en los lineamientos del estándar **IEEE 1058**, este documento establece una hoja de ruta clara para el ciclo de vida del software. Define el alcance del trabajo, la estimación de esfuerzo, la gestión de riesgos y la arquitectura base necesaria para materializar la plataforma.

Este plan está concebido como un "documento vivo". Su evolución acompañará el progreso del proyecto, iterando desde una fase inicial de estricta especificación arquitectónica y documental, hasta la fase operativa de desarrollo y entrega del Producto Mínimo Viable (MVP). De esta manera, el SDP actúa como la principal herramienta para la toma de decisiones, garantizando la trazabilidad, la calidad y la mitigación proactiva de impedimentos a lo largo de todo el ciclo de desarrollo

CONTENIDOS

1. Introducción	4
1.1 Visión General del Proyecto	4
1.2 Prestaciones del Proyecto	4
1.3 Evolución del Plan de Gestión del Proyecto del Software	6
1.4 Materiales de Referencia	7
1.5 Definiciones y Acrónimos	7
2. Organización del Proyecto	9
2.1 Modelo del Proceso	9
2.2 Estructura Organizacional	13
2.3 Interfaces y Límites Organizacionales	13
2.4 Responsabilidades del Proyecto	14
3. Proceso Directivo	15
3.1 Objetivos y Prioridades de la Gestión	15
3.2 Supuestos, Dependencias y Limitaciones	16
3.3 Gestión de Riesgos	18
3.4 Mecanismos de Control y Seguimiento.	33
3.5 Plan de Dotación del Personal	35
4. Proceso Técnico	36
4.1 Métodos, Herramientas y Técnicas	36
4.1.2 Herramientas de Software	36
4.1.3 Metodologías, Lenguajes y Estándares de Codificación	36
4.1.4 Técnicas de Verificación y Validación	37
4.2 Documentación del Software	37
4.3 Funciones Soporte del Proyecto	39
5. Paquetes de Trabajo, Calendario y Presupuesto.	42
5.1 Paquetes de Trabajo	42
5.2 Dependencias	54
5.3 Recursos Necesitados	60
5.4 Asignación del Presupuesto y de los Recursos	62
5.5 Calendario	64
5.6 Estimación de esfuerzo	65
6. Componentes Adicionales	76
7. Índice	78
8. Apéndices	79

1. INTRODUCCIÓN

1.1 Visión General del Proyecto

ConstructING es una plataforma móvil diseñada para resolver la asimetría de información y la falta de transparencia entre profesionales de la construcción (Ingenieros, MMO, Arquitectos) y propietarios durante el desarrollo de obras o reformas. El sistema permite la certificación de hitos técnicos mediante evidencia multimedia georreferenciada y firmas digitales, bajo una arquitectura offline-first que garantiza la operatividad en zonas de baja conectividad. Su propósito principal es eliminar la necesidad de visitas presenciales constantes del propietario y otorgar respaldo legal y técnico al trabajo del profesional, excluyendo deliberadamente la gestión contable o el control de presentismo para mantener el foco en la auditoría técnica inalterable.

El proyecto se divide en dos grandes etapas: Fase 1 (Ingeniería de Requisitos y Diseño - 1° Cuatrimestre) y Fase 2 (Desarrollo y Despliegue del MVP - 2° Cuatrimestre). Al tratarse de un proyecto académico individual, los recursos humanos se limitan a un único desarrollador asumiendo roles integrales (Project Manager, Arquitecto, QA y Dev). El presupuesto se considera de costo cero, con un calendario fijado para su presentación final en diciembre de 2026.

1.2 Prestaciones del Proyecto

A continuación, se especifican los entregables formales del ecosistema **ConstructING**. El ciclo de vida del proyecto combina la disciplina de documentación de la arquitectura técnica (Fase 1) con la flexibilidad operativa de **Scrum** y métricas ágiles para la construcción del MVP (Fase 2).

1. Paquete de Documentación Técnica y de Gestión (Fase 1 - Ingeniería de Requisitos y Diseño)

- **Contenido:**
 - Project Charter del proyecto.
 - Especificación de Requisitos de Software (ERS/SRS basado en IEEE 830).
 - Anexo de Especificación de Casos de Uso (CdU) detallados.
 - Plan de Desarrollo de Software (SDP basado en IEEE 1058) actualizado al cierre de la fase.
 - Modelos y Diccionario de Datos (Modelo Entidad-Relación y Modelo Relacional).
 - Prototipos de Interfaz de Usuario de alta fidelidad (UI Mockups en Figma).
- **Fecha de entrega:** Fin del 1° Cuatrimestre (Julio 2026).
- **Método de entrega:** Envío digital de archivos PDF inalterables y enlaces a los artefactos interactivos de diseño (Figma) por plataforma Institucional y repositorio en la nube.
- **Cantidad:** 1 set documental completo y unificado.

2. Artefactos de Gestión y Métricas de Sprints (Fase 2 - Desarrollo Ágil bajo Scrum)

- **Contenido:**

- **Product Backlog:** Listado priorizado de Historias de Usuario correspondientes a los módulos de Usuarios (RBAC), Obras, Evidencia Visual con Georreferenciación, y Certificación con Firma Digital y Hash (SHA-256).
- **Sprint Planning y Sprint Backlogs:** Planificación y alcance cerrado para cada uno de los bloques iterativos de desarrollo (Sprints de 2 o 3 semanas de duración).
- **Burndown Charts de los Sprints:** Gráficos de trabajo pendiente actualizados de manera diaria que evidencien el ritmo de desarrollo real contra la línea ideal de "quemado" de tareas, permitiendo el control visual de desviaciones en el alcance o en el esfuerzo estimado.
- **Fecha de entrega:** De manera incremental durante el 2° Cuatrimestre; consolidación final en Noviembre/Diciembre 2026.
- **Método de entrega:** Tableros de gestión digital (Jira / GitHub Projects) accesibles para los evaluadores y reportes analíticos consolidados.
- **Cantidad:** 1 tablero activo y reportes analíticos por cada Sprint ejecutado.

3. Repositorio de Código Fuente y Control de Versiones (Fase 2 - Construcción)

- **Contenido:** Código fuente del ecosistema estructurado bajo los principios del Proceso Unificado en cuanto a calidad de diseño:
 - Frontend mobile desarrollado en Flutter con arquitectura *offline-first*.
 - Backend/API robusto desarrollado en NestJS.
 - Scripts de migración, definición de esquemas (DDL) e inicialización de la base de datos relacional PostgreSQL.
- **Fecha de entrega:** Cierre de la etapa de desarrollo (Noviembre 2026).
- **Método de entrega:** Acceso a repositorio privado/público en GitHub con un historial de *commits* limpio, trazable y segmentado por ramas de características (*feature branches*).
- **Cantidad:** 1 repositorio centralizado con la rama **main** en estado estable y desplegable.

4. Producto Mínimo Viable (MVP) Operativo y Desplegado (Fase 2 - Cierre)

- **Contenido:** Solución de software funcional de punta a punta. Incluye la base de datos y la API backend desplegadas en infraestructura en la nube (AWS/Supabase), y la aplicación cliente ejecutable.
- **Fecha de entrega:** Diciembre 2026.
- **Método de entrega:**
 - **Mobile:** Archivo binario instalable (APK para Android) distribuido mediante enlace de descarga segura (Google Drive).

- **Servicios de Servidor:** Enlaces HTTPS activos a los entornos de producción del backend.
- **Cantidad:** 1 instalador APK ejecutable y 1 ecosistema en la nube en pleno funcionamiento.

5. Material de Presentación y Defensa Final (Fase 2 - Cierre Académico)

- **Contenido:** Diapositivas de soporte técnico-comercial, guión de demostración funcional en vivo (*Live Demo*) de los Casos de Uso críticos, y balance final comparativo del proyecto (Riesgos mitigados vs. Planificados).
- **Fecha de entrega:** Diciembre 2026 (Día fijado para la defensa de la materia).
- **Método de entrega:** Exposición síncrona asistida por material visual interactivo.
- **Cantidad:** 1 presentación digital multimedia

1.3 Evolución del Plan de Gestión del Proyecto del Software

Versión	Autor (es) Primario (s)	Descripción de la Versión	Fecha Esperada
Borrador	Nicolás Sebastián del Río	Borrador inicial centrado en las bases estructurales, el análisis de riesgos predictivos y la definición de alcances para la Fase 1.	Mayo 2026
Preliminar	Nicolás Sebastián del Río	Segundo borrador. Incorporación de refinamientos en la Especificación de Requisitos, modelos relacionales y mitigación de riesgos tras la revisión técnica de la cátedra.	Julio 2026
Revisión 1	Nicolás Sebastián del Río	Actualización operativa (Inicio Fase 2). Se incorporan el <i>Product Backlog</i> inicial y la planificación de los primeros Sprints de desarrollo bajo Scrum.	Agosto 2026
Revisión 2	Nicolás Sebastián del Río	Actualización ágil. Se anexan los primeros <i>Burndown Charts</i> , métricas de velocidad empíricas, y ajustes técnicos sobre el desarrollo <i>offline-first</i> y API REST.	Octubre 2026

Versión	Autor (es) Primario (s)	Descripción de la Versión	Fecha Esperada
Final	Nicolás Sebastián del Río	Versión definitiva bajo estricto control de cambios. Refleja el MVP completado, el balance final del alcance y el reporte de riesgos mitigados durante el cierre del proyecto.	Noviembre 2026

1.4 Materiales de Referencia

A continuación, se listan los documentos, estándares y referencias bibliográficas en los cuales se fundamenta la planificación y ejecución de este proyecto:

- **McConnell, S. (1998). *Software Project Survival Guide*. Microsoft Press.** El presente plan es una adaptación del Software Development Plan (SDP) propuesto en esta obra.
- **IEEE Std 1058-1998. *Estándar IEEE para Planes de Gestión de Proyectos de Software*** (Base estructural de este documento SDP).
- **IEEE Std 830-1998. *Práctica Recomendada por IEEE para las Especificaciones de Requisitos de Software*** (Base para la redacción de la ERS).
- **Documentación Interna: *Project Proposal de ConstructING*.** (Documento elaborado en la Etapa 1 del proyecto).
- **Documentación Interna: *Especificación de Requisitos de Software (ERS)* y *Anexo de Casos de Uso*.** (Línea base del alcance elaborada en la Fase 1).
- **Documentación Interna: *Propósito del Sistema (PdS)*.** (Definición principal del ecosistema).
- **Documentación Interna Anexa: *SIP 2026 - P02 - IEE Anexo MatrizRiesgos - DEL RIO Nicolas Sebastian.xlsx*.** (Modelo analítico de la Matriz de Probabilidad-Impacto y Registro Histórico de Riesgos Inherentes y Residuales para el ecosistema ConstructING).

1.5 Definiciones y Acrónimos

Acrónimos:

- **API:** Application Programming Interface (Interfaz de Programación de Aplicaciones).
- **APK:** Android Package Kit (Archivo binario ejecutable e instalable para el ecosistema móvil Android).

- **AWS:** Amazon Web Services (Proveedor de servicios de infraestructura en la nube).
- **CdU:** Casos de Uso.
- **DDL:** Data Definition Language (Lenguaje de Definición de Datos, utilizado en los scripts de la base de datos PostgreSQL).
- **DER:** Diagrama Entidad-Relación (Artefacto de arquitectura de datos).
- **EDT / WBS:** Estructura de Descomposición del Trabajo / Work Breakdown Structure.
- **ERS / SRS:** Especificación de Requisitos de Software / Software Requirements Specification.
- **MMO:** Maestro Mayor de Obras (Profesional técnico del rubro de la construcción y usuario de la plataforma).
- **MVP:** Producto Mínimo Viable (Minimum Viable Product).
- **QA:** Quality Assurance (Aseguramiento de la Calidad).
- **RBAC:** Role-Based Access Control (Control de Acceso Basado en Roles).
- **SCM:** Software Configuration Management (Gestión de la Configuración del Software).
- **SDP:** Software Development Plan (Plan de Desarrollo del Software).
- **UI / UX:** User Interface / User Experience (Interfaz de Usuario / Experiencia de Usuario).

Definiciones:

- **Burndown Chart:** Gráfico de quemado o trabajo pendiente actualizado de manera diaria que evidencia el ritmo de desarrollo real contra la línea ideal de avance en un Sprint.
- **Greenfield:** Proyecto desarrollado completamente desde cero, sin utilización, modificación ni refactorización de código base preexistente o de sistemas ajenos.
- **Hash (SHA-256):** Algoritmo criptográfico utilizado para generar una cadena de texto inalterable que actúa como sello de integridad y firma digital para las evidencias documentales (archivos PDF) generadas por el sistema.
- **Offline-first:** Arquitectura de software diseñada para garantizar la operatividad de la aplicación móvil en zonas de nula o baja conectividad. Esta arquitectura almacena localmente los datos de auditoría y los sincroniza a la nube en cuanto la red se restablece.
- **Product Backlog:** Listado dinámico y priorizado de Historias de Usuario correspondientes a los módulos a desarrollar en el MVP.
- **Scope Creep:** Riesgo de proyecto que implica la ampliación, crecimiento o cambio no planificado del alcance del software una vez que los requerimientos base ya han sido aprobados.

- **Sprint:** Ciclos o bloques iterativos de desarrollo bajo el marco Scrum, de duración preestablecida (2 a 3 semanas), planificados para construir y validar funcionalidades incrementales del software

2. ORGANIZACIÓN DEL PROYECTO

2.1 Modelo del Proceso

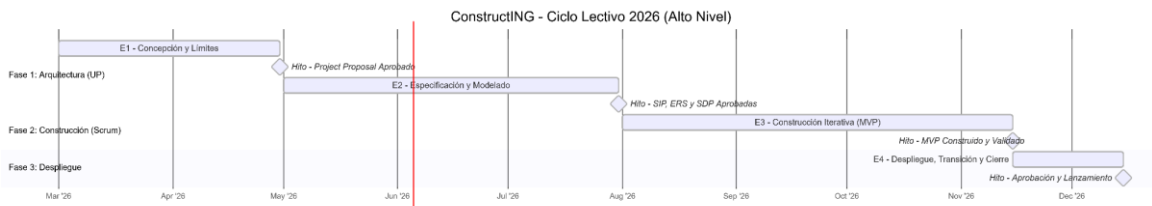
El proyecto **ConstructING** adopta un modelo de ciclo de vida **híbrido y evolutivo**, separado en dos fases claves de la siguiente manera;

- **Fase de Ingeniería de Requisitos y Arquitectura (1er Cuatrimestre):** Adopta las disciplinas de las fases de Inicio y Elaboración del **Proceso Unificado (UP)**. El enfoque es puramente secuencial y dirigido por la arquitectura y los riesgos, prohibiendo la generación de código de producción hasta consolidar las bases lógicas.
- **Fase de Construcción del MVP (2do Cuatrimestre):** Transiciona hacia un enfoque **Ágil e Incremental utilizando el marco Scrum**. El alcance se gestiona mediante un *Product Backlog* dinámico y se ejecuta en bloques temporales cerrados (*Sprints*), permitiendo la evolución del software mediante prototipos operativos funcionales.

Etapas más importantes del proyecto (Calendario de alto nivel)

El cronograma del proyecto se distribuye a lo largo del ciclo lectivo 2026 bajo el siguiente esquema descriptivo de hitos:

- **Etapas 1: Concepción, Visión y Límites (Marzo – Abril 2026):** Definición del problema, justificación del ecosistema móvil y delineación de fronteras (foco exclusivo en auditoría técnica, excluyendo contabilidad y presentismo). *Hito de salida: Project Proposal aprobado.*
- **Etapas 2: Especificación y Modelado Arquitectónico (Mayo – Julio 2026):** Elicitación detallada de requerimientos, diseño del Modelo Entidad-Relación y Relacional (PostgreSQL), especificación de Casos de Uso y diseño de la interfaz de usuario de alta fidelidad (UI Figma). *Hito de salida: SIP, ERS y SDP Aprobadas.*
- **Etapas 3: Construcción Iterativa e Incremental - MVP (Agosto – Noviembre 2026):** Desarrollo del código fuente dividiendo el sistema en Sprints de 2 a 3 semanas. Implementación de la arquitectura *offline-first* (Flutter), API Backend (NestJS) y persistencia inalterable de evidencias con georreferenciación y firmas digitales (Hash SHA-256). *Hito de salida: MVP funcional y Burndown Charts actualizados.*
- **Etapas 4: Despliegue, Transición y Cierre (Noviembre – Diciembre 2026):** Migración y puesta en producción del backend (AWS/Supabase), generación del paquete instalable móvil (APK Android) y preparación de la defensa final del proyecto. *Hito de salida: Aprobación y Lanzamiento.*



Nombre del Producto de Trabajo	Fecha Planeada de Finalización	¿Colocado Bajo el Control de Cambios?	¿Entregado al Cliente? (Cátedra)	Personas que Deben Firmar el Producto de Trabajo
Declaración de la Visión (Project Charter-Project Proposal)	Abril 2026	Sí	No	Nicolás del Rio (Gerente / Ing. Líder)
Plan de Desarrollo del Software (SDP)	Mayo 2026	Sí	Sí	Nicolás del Rio (Gerente de Proyecto)
Lista de los 10 Riesgos más importantes	Mayo 2026	Sí	No	Nicolás del Rio (Gerente de Proyecto)

Plan de Control de Cambios	Junio 2026	Sí	No	Nicolás del Rio (Gerente de Proyecto)
Manual de Requisitos (ERS / SRS IEEE 830)	Julio 2026	Sí	Sí	Nicolás del Rio (Ingeniero Líder)
Guía de Estilo de la Interfaz de Usuario - Anexo ERS	Julio 2026	Sí	Sí	Nicolás del Rio (Diseñador UI/UX)
Arquitectura del Software (Modelos de Datos / DER)	Julio 2026	Sí	No	Nicolás del Rio (Arquitecto de Software)
Plan de Aseguramiento de la Calidad (QA)	Agosto 2026	Sí	No	Nicolás del Rio (QA Líder)
Estándar de Codificación	Agosto 2026	No	No	Nicolás del Rio (Ingeniero Líder)

Plan de Entrega por Etapas (<i>Product Backlog</i>)	Agosto 2026	Sí	No	Nicolás del Rio (<i>Gerente de Proyecto</i>)
Etapas Individual de Planes (<i>Sprint Planning</i>)	Septiembre 2026	Sí	No	Nicolás del Rio (<i>Ingeniero Líder</i>)
Diseño Detallado de Casos de uso (<i>Anexo ERS</i>)	Septiembre 2026	Sí	Sí	Nicolás del Rio (<i>Ingeniero Líder</i>)
Planes de Construcción del Software	Octubre 2026	Sí	No	Nicolás del Rio (<i>Ingeniero Líder</i>)
Código Fuente Final del Software	Noviembre 2026	Sí	Sí	Nicolás del Rio (<i>Ingeniero Líder</i>)
Despliegue de Documentos (<i>Manual AWS/Supabase</i>)	Noviembre 2026	Sí	No	Nicolás del Rio (<i>Documentación Principal</i>)

Lista de Control de Lanzamiento	Noviembre 2026	No	No	Nicolás del Rio (QA Líder)
Formulario de Aprobación del Lanzamiento	Diciembre 2026	Sí	Sí	Nicolás del Rio (PM), Docente Cátedra (Client)
Registro del Proyecto del Software (Logs Jira/GitHub)	Diciembre 2026	No	Sí	Nicolás del Rio (Gerente de Proyecto)
Documento Histórico (Burndown Charts Consolidados)	Diciembre 2026	No	Sí	Nicolás del Rio (Gerente de Proyecto)

2.2 Estructura Organizacional

Al tratarse de un Proyecto desarrollado de manera individual, la estructura organizacional interna del equipo es plana (flat). Un único integrante asume la totalidad de los roles operativos, técnicos y de gestión. La máxima autoridad de decisión sobre la arquitectura y el desarrollo recae sobre el alumno (quien actúa como *Project Lead* y Arquitecto), el cual rinde cuentas directamente al cuerpo docente, que asume el rol externo de "Cliente Auditor" o "Stakeholder Evaluador".

2.3 Interfaces y Límites Organizacionales

ConstructING es un proyecto académico desarrollado de forma estrictamente individual en el marco de la materia Seminario de Integración Profesional, por lo que no se identifican interfaces organizacionales formales en el sentido tradicional de la industria. No existe una organización cliente externa, proveedores, subcontratistas, ni equipos separados de aseguramiento de calidad (QA), control de configuración, documentación o soporte al

usuario final. Todas las responsabilidades técnicas y de gestión —incluyendo la ingeniería de requerimientos, el diseño de la arquitectura del ecosistema móvil y backend, la codificación y las pruebas— son asumidas íntegramente por el alumno desarrollador (Nicolás Sebastián del Río).

No obstante, cabe destacar que existe una interfaz de comunicación crítica con el ecosistema académico. El docente responsable de la cátedra (Lic. Christian López Pasarón), actúa como máximo evaluador del proyecto. Su participación es análoga a la de un *Cliente* o *Product Owner* externo, ya que interviene en las instancias de revisión para validar los entregables documentales en la fase de planeación (SRS, Casos de Uso, Modelo Relacional, Prototipado) y, posteriormente, verificará los incrementos de software durante el desarrollo del MVP. Su retroalimentación es vinculante y condiciona directamente la planificación, la validación de la lógica arquitectónica y la continuidad general del proyecto.

2.4 Responsabilidades del Proyecto

Responsabilidad	Persona (s) Responsable (s)
Gestión Global del Proyecto	Nicolás del Río
Gestión de Ingeniería	Nicolás del Río
Gestión del Aseguramiento de Calidad	Nicolás del Río
Gestión de la Documentación del Usuario final.	Nicolás del Río
Requerimientos de Desarrollo	Nicolás del Río
Arquitectura del Software	Nicolás del Río
Auto-revisión técnica	Nicolás del Río
Diseño de Interfaz de Usuario	Nicolás del Río
Desarrollo de Módulos del Sistema	Nicolás del Río
Ejecución de Pruebas y Validación	Nicolás del Río
Elaboración de Documentación Técnica	Nicolás del Río

3. PROCESO DIRECTIVO

3.1 Objetivos y Prioridades de la Gestión

La filosofía de gestión de **ConstructING** se fundamenta en la transparencia radical, la mitigación temprana de riesgos técnicos y el control empírico del avance. La meta principal es asegurar la entrega de un Producto Mínimo Viable (MVP) completamente funcional, robusto y desplegado al cierre del ciclo lectivo 2026, evitando la degradación de la arquitectura definida en la Fase 1 durante la velocidad de los Sprints de la Fase 2.

2. Tipo de reportes de estado

Al tratarse de un proyecto de desarrollo individual, los mecanismos de reporte se adaptan estructuralmente para servir como evidencia auditable ante la cátedra:

- **Reportes Incrementales (Cierre de Sprint):** Al finalizar cada ciclo de desarrollo (2 a 3 semanas), se generará un estado del *Product Backlog* detallando las Historias de Usuario completadas contra las comprometidas.
- **Métricas de Control Visual (Burndown Chart):** El artefacto principal de reporte diario será el gráfico de quemado. Este reporte visual contrastará la línea de esfuerzo estimado contra el trabajo real remanente, identificando desviaciones de forma inmediata para forzar acciones correctivas antes del cierre de cada iteración.

3. Prioridades relativas (Funcionalidad, Calendario y Presupuesto)

La matriz de prioridades del proyecto se define bajo un esquema estricto de restricciones académicas (*Triple Restricción*):

1. **Calendario (Prioridad Máxima - Inalterable):** La fecha de finalización y defensa (Diciembre 2026) está fijada por el calendario universitario. No existe flexibilidad temporal externa.
2. **Presupuesto (Fijo - Costo Cero en Mano de Obra):** Al ser un proyecto unipersonal, el presupuesto financiero está acotado estrictamente a los niveles gratuitos o de bajo costo de la infraestructura cloud (AWS / Supabase), por lo que actúa como una restricción dura.
3. **Funcionalidad (Variable / Ajustable):** Para garantizar el cumplimiento del calendario bajo un presupuesto acotado, **el alcance es la variable de ajuste**. Si el *Burndown Chart* demuestra un retraso persistente debido a la complejidad de la arquitectura *offline-first*, las funcionalidades secundarias se renegociarán o se pasarán a futuras versiones (respetando siempre el núcleo inalterable: la auditoría técnica, las evidencias georreferenciadas y la firma con hash de integridad).

4. Procedimientos para la gestión de riesgos

El proyecto adopta un enfoque proactivo de gestión de riesgos basado en los lineamientos de la cátedra:

- **Identificación y Evaluación:** Se mantiene una lista viva de riesgos técnicos y de gestión (por ejemplo, fallas en la sincronización local o curvas de aprendizaje de los frameworks).
- **Monitoreo Continuo:** En cada planificación de Sprint se evaluará si las tareas comprometidas disparan algún riesgo crítico.
- **Mitigación Ágil:** Si un riesgo se materializa, impactando negativamente en la curva real del *Burndown Chart*, se ejecutarán de inmediato planes de contingencia preestablecidos o se reducirá el alcance no esencial del Sprint.

5. Enfoque para la adquisición de software de terceros y modificación de software existente

- **Adquisición de Terceros:** El ecosistema se construirá utilizando tecnologías de código abierto (*Open Source*) consolidadas en la industria (Flutter, NestJS, PostgreSQL). La inclusión de dependencias de terceros (librerías/paquetes en Pub.dev o npm) se limitará estrictamente a aquellas que cuenten con soporte activo, licencias permisivas (MIT/Apache 2.0) y que no comprometan la seguridad de la transmisión de datos biométricos y hashes.
- **Software Existente:** El proyecto se desarrolla completamente desde cero (*Greenfield*), por lo que no se contempla la modificación, refactorización o utilización de código base preexistente de sistemas ajenos.

3.2 Supuestos, Dependencias y Limitaciones

3.2.1 Supuestos

Los planes y estimaciones de este proyecto se basan en el cumplimiento de los siguientes supuestos lógicos:

- **Estabilidad de Requisitos:** Se asume que la Especificación de Requisitos de Software (ERS Versión 0.8.0) consolidada al cierre de la Fase 1 es lo suficientemente sólida y madura para guiar el desarrollo de la Fase 2, minimizando cambios drásticos en el alcance intermedio.
- **Disponibilidad de Infraestructura Gratuita:** Se asume que las plataformas de servicios en la nube elegidas (AWS / Supabase) mantendrán la vigencia de sus capas gratuitas (*Free Tiers*) o créditos académicos durante todo el ciclo de desarrollo del MVP.
- **Capacidad de Dedicación Unipersonal:** Se asume que el único integrante del equipo mantendrá la disponibilidad de tiempo y la salud óptima para cumplir con la carga horaria semanal estimada para los Sprints de la Fase 2.

- **Continuidad Académica:** Se asume que el calendario académico y las condiciones de regularidad dictadas por la Universidad del Salvador (USAL) para el ciclo lectivo 2026 se mantendrán sin alteraciones externas de fuerza mayor.

3.2.2 Dependencias

El éxito en la ejecución del cronograma está sujeto a las siguientes dependencias críticas, tanto internas como externas:

- **Aprobación Académica de la Fase 1:** El inicio de la Fase 2 (Construcción del MVP) depende estrictamente de la aprobación de la línea base arquitectónica y la documentación técnica por parte de la cátedra al finalizar el primer cuatrimestre.
- **Ecosistema de Software de Terceros:** La implementación de la arquitectura *offline-first* y la firma digital depende de la disponibilidad, compatibilidad y estabilidad de las librerías de código abierto seleccionadas para Flutter y NestJS (motores de persistencia local, paquetes de georreferenciación GPS y módulos criptográficos).
- **Conectividad para Despliegue:** Aunque la aplicación móvil está diseñada bajo el paradigma *offline-first* para el usuario final, el proceso de integración continua, despliegue del backend y pruebas de sincronización cloud dependen de una conectividad a internet estable en el entorno de desarrollo.

3.2.3 Limitaciones (Restricciones)

Las restricciones duras que delimitan y condicionan las decisiones de gestión y técnicas del proyecto son:

- **Calendario (Tiempo):** La limitación más crítica. La fecha final de entrega y defensa del proyecto está fijada para diciembre de 2026. No existe posibilidad de extensión temporal, lo que obliga a un control estricto de los Sprints mediante los *Burndown Charts*.
- **Presupuesto (Financiero):** El presupuesto asignado para la adquisición de software, licencias o hardware es de \$0 USD. Cualquier costo derivado del uso de servidores cloud que exceda las capas gratuitas debe ser mitigado mediante optimización de código y diseño arquitectónico eficiente.
- **Recursos Humanos (Esfuerzo):** Al ser un proyecto individual, la velocidad de desarrollo está limitada estrictamente a la capacidad de producción de una sola persona. No es posible mitigar retrasos sumando personal al equipo (Ley de Brooks), por lo que ante desvíos se deberá ajustar la funcionalidad.
- **Funcionalidad (Alcance):** El alcance está restringido estrictamente a las fronteras definidas en la documentación presentada durante el desarrollo del primer cuatrimestre. Queda totalmente excluida la gestión contable, compra de materiales, control de presentismo y mensajería interna, priorizando exclusivamente el módulo de auditoría técnica y evidencias fehacientes.

- **Calidad y Normativas:** El producto final debe cumplir de forma obligatoria con el estándar de documentación IEEE exigido por la asignatura y garantizar la inalterabilidad de los archivos PDF generados mediante firmas digitales y hashes de integridad (SHA-256).

3.3 Gestión de Riesgos

El proyecto ConstructING adopta un enfoque de gestión de riesgos continuo y proactivo, entendiendo al riesgo como la probabilidad de que una amenaza interna o externa afecte el cronograma, la calidad o la viabilidad del MVP.

Rastreo y Monitoreo de Riesgos: La evolución de los riesgos será rastreada y monitoreada de forma iterativa mediante los siguientes mecanismos de control:

- **Instancias de Evaluación Ágil:** La Lista de los 10 Riesgos se revisará obligatoriamente durante el *Sprint Planning* (para evaluar si las Historias de Usuario seleccionadas exponen al proyecto a un nuevo riesgo) y en la *Sprint Retrospective* (para documentar si un riesgo se materializó y medir la efectividad de la mitigación).
- **Sistema de Alerta Temprana:** El *Burndown Chart* actuará como el principal indicador visual de riesgos materializados. Cualquier desviación sostenida de la curva de trabajo real por encima de la línea ideal indicará un problema subyacente de estimación, un bloqueo tecnológico o una carga de trabajo externa no programada, forzando la activación de los planes de mitigación.

Los datos detallados a continuación se encuentran consolidados y parametrizados en el archivo anexo '**SIP 2026 - P02 - IEE Anexo MatrizRiesgos - DEL RIO Nicolas Sebastian.xlsx**'. Este artefacto interactivo opera como el registro maestro de riesgos de ConstructING, automatizando el cálculo de criticidades, la matriz de Probabilidad-Impacto y la evolución del riesgo inherente al residual como soporte para los *Sprints* de la Fase 2.

A continuación, se detalla la matriz cuantitativa (utilizando la escala de Fibonacci 1, 2, 3, 5, 8 para Probabilidad e Impacto) con los 10 riesgos críticos identificados para ConstructING, estructurados en su estado Inherente y Residual:

1. Mala estimación del esfuerzo necesario

Riesgo Inherente

ID	Categoría	Descripción	Probabilidad	Impacto	Riesgo	Plan de Mitigación
RO 1	Planificación	Subestimar el tiempo requerido para configurar la sincronización de la DB local con la nube (Supabase) y el manejo de evidencias.	Probable (5)	Alto (5)	25	Utilizar estimación por Puntos de Historia, apoyarse en el Burndown Chart para seguimiento diario y agregar un "buffer" de contingencia del 20% al cronograma de la Fase 2.

Riesgo Residual

ID	Categoría	Descripción	Probabilidad	Impacto	Riesgo

R01	Planificación	Subestimar el tiempo requerido para configurar la sincronización de la DB local con la nube (Supabase) y el manejo de evidencias.	Posible (3)	Significativo (3)	9
-----	---------------	---	-------------	-------------------	---

2. Inexperiencia con la tecnología

Riesgo Inherente

ID	Categoría	Descripción	Probabilidad	Impacto	Riesgo	Plan de Mitigación
R02	Tecnológico	Curva de aprendizaje alta o bloqueo técnico al implementar la captura biométrica en pantalla y la generación de hashes (SHA-256) en PDFs.	Posible (3)	Alto (5)	15	Destinar semanas específicas en la Fase 1 para desarrollar Pruebas de Concepto (PoC) sobre firmas digitales, mitigando la incertidumbre arquitectónica antes de

iniciar el MVP.

ID	Categoría	Descripción	Probabilidad	Impacto	Riesgo
R02	Tecnológico	Curva de aprendizaje alta o bloqueo técnico al implementar la captura biométrica en pantalla y la generación de hashes (SHA-256) en PDFs.	Improbable (2)	Significativo (3)	6

3. Enfermedades (Indisponibilidad del Recurso)

Riesgo Inherente

ID	Categoría	Descripción	Probabilidad	Impacto	Riesgo	Plan de Mitigación
RO3	Recurso Humano	Problemas de salud del desarrollador que impidan avanzar con la codificación o documentación durante semanas críticas del cuatrimestre.	Posible (3)	Extremo (8)	24	Mantener toda la documentación y código centralizado en la nube (GitHub). Planificar los sprints con holgura para poder absorber hasta 10 días de inactividad sin comprometer el hito final.

Riesgo Residual

ID	Categoría	Descripción	Probabilidad	Impacto	Riesgo

R03	Recurso Humano	Problemas de salud del desarrollador que impidan avanzar con la codificación o documentación durante semanas críticas del cuatrimestre.	Posible (3)	Significativo (3)	9
-----	----------------	---	-------------	-------------------	---

4. Cambios en el proyecto (Scope Creep)

Riesgo Inherente

ID	Categoría	Descripción	Probabilidad	Impacto	Riesgo	Plan de Mitigación
R04	Requerimientos	Ampliación no planificada del alcance del MVP durante las revisiones con la cátedra o por sobre-ambición propia (ej. agregar gestión de	Probable (5)	Alto (5)	25	Limitar estrictamente el alcance funcional en la ERS y el documento PdS. Congelar requerimientos al fin de la Fase 1. Las nuevas ideas se documentarán en el

		presupuest os).				Backlog para "futuras versiones".
--	--	--------------------	--	--	--	--

Riesgo Residual

ID	Categoría	Descripción	Probabilidad	Impacto	Riesgo
RO 4	Requerimiento s	Ampliación no planificada del alcance del MVP durante las revisiones con la cátedra o por sobre- ambición propia (ej. agregar gestión de presupuestos) .	Improbable (2)	Significativ o (3)	6

5. Diseño inadecuado

Riesgo Inherente

ID	Categoría	Descripción	Probabilidad	Impacto	Riesgo	Plan de Mitigación
R05	Diseño / UX	Diseñar un flujo de usuario complejo que confunda al Propietario o entorpezca el trabajo rápido del Profesional en el entorno de obra.	Posible (3)	Alto (5)	15	Realizar wireframes y prototipos interactivos completos en Figma durante la Fase 1. Validar la usabilidad de las interfaces con terceros antes de iniciar la etapa de codificación frontend.

Riesgo Residual

ID	Categoría	Descripción	Probabilidad	Impacto	Riesgo

R05	Diseño / UX	Diseñar un flujo de usuario complejo que confunda al Propietario o entorpezca el trabajo rápido del Profesional en el entorno de obra.	Raro (1)	Bajo (2)	2
-----	-------------	--	----------	----------	---

6. Fallas en los servicios básicos

Riesgo Inherente

ID	Categoría	Descripción	Probabilidad	Impacto	Riesgo	Plan de Mitigación
R06	Infraestructura	Caída, interrupción temporal o cambios de política en las capas gratuitas de los servicios Cloud (Supabase, AWS) durante el desarrollo.	Improbable (2)	Extremo (8)	16	Justificar y robustecer la arquitectura Offline-First de la aplicación; el sistema móvil debe ser capaz de almacenar evidencias y registros localmente hasta que la infraestructura en la

Riesgo Residual

7. Supuestos no válidos

Riesgo Inherente

ID	Categoría	Descripción	Probabilidad	Impacto	Riesgo	Plan de Mitigación
RO 7	Negocio / Legal	Asumir erróneamente que las certificaciones de la app tendrán validez legal automática y definitiva ante un pleito civil, generando rechazo en la defensa final.	Probable (5)	Extremo (8)	40	Redactar un apartado claro en la documentación indicando que el sistema brinda "trazabilidad inalterable de evidencias como soporte técnico", pero no reemplaza marcos legales notariales vigentes.

Riesgo Residual

ID	Categoría	Descripción	Probabilidad	Impacto	Riesgo

R07	Negocio / Legal	Asumir erróneamente que las certificaciones de la app tendrán validez legal automática y definitiva ante un pleito civil, generando rechazo en la defensa final.	Improbable (2)	Bajo (2)	4
-----	-----------------	--	----------------	----------	---

8. Lentitud en una toma de decisiones

Riesgo Inherente

ID	Categoría	Descripción	Probabilidad	Impacto	Riesgo	Plan de Mitigación
R08	Gestión	Demoras importantes en recibir feedback o aprobaciones documentales por parte de la tutoría académica, retrasando el avance entre fases.	Posible (3)	Significativo (3)	9	Establecer un canal fluido y un cronograma de entregas pactadas. Paralelizar tareas de investigación técnica mientras se aguardan las revisiones documentales

						es para no detener el progreso.
--	--	--	--	--	--	---------------------------------

Riesgo Residual

ID	Categoría	Descripción	Probabilidad	Impacto	Riesgo
R08	Gestión	Demoras importantes en recibir feedback o aprobaciones documentales por parte de la tutoría académica, retrasando el avance entre fases.	Improbable (2)	Bajo (2)	4

9. Tareas no programadas (Carga paralela)

Riesgo Inherente

ID	Categoría	Descripción	Probabilidad	Impacto	Riesgo	Plan de Mitigación
RO9	Proyecto	Superposición de entregas críticas o exigencias pesadas de otras materias de la carrera que resten horas de disponibilidad al desarrollo del MVP.	Probable (5)	Alto (5)	25	Utilizar la Estructura de Descomposición del Trabajo (EDT) para planificar los hitos de desarrollo más pesados en épocas "valle" (alejadas de las semanas de exámenes parciales/finales).

Riesgo Residual

ID	Categoría	Descripción	Probabilidad	Impacto	Riesgo

R09	Proyecto	Superposición de entregas críticas o exigencias pesadas de otras materias de la carrera que resten horas de disponibilidad al desarrollo del MVP.	Posible (3)	Significativo (3)	9
-----	----------	---	-------------	-------------------	---

Riesgo 10: Resistencia al cambio

Riesgo Inherente

ID	Categoría	Descripción	Probabilidad	Impacto	Riesgo	Plan de Mitigación
----	-----------	-------------	--------------	---------	--------	--------------------

R10	Negocio / Adopción	Resistencia por parte de los profesionales de obra a adoptar una nueva herramienta formal, percibiéndola como burocracia que retrasa su trabajo en campo.	Probable (5)	Alto (5)	25	Cumplimiento estricto del atributo de usabilidad (RNF_U_01 y RNF_U_02): curva de aprendizaje menor a 15 minutos y registro de evidencia en máximo 3 interacciones. El diseño (UI/UX) debe priorizar la inmediatez operativa.
-----	--------------------	---	--------------	----------	----	--

Riesgo Residual

ID	Categoría	Descripción	Probabilidad	Impacto	Riesgo
----	-----------	-------------	--------------	---------	--------

R10	Negocio / Adopción	Resistencia por parte de los profesionales de obra a adoptar una nueva herramienta formal, percibiéndola como burocracia que retrasa su trabajo en campo.	Posible (3)	Significativo (3)	9
-----	--------------------	---	-------------	-------------------	---

3.4 Mecanismos de Control y Seguimiento.

Para garantizar que el proyecto se mantenga alineado con los objetivos de la cátedra y mitigar desvíos de manera oportuna, se establecen mecanismos métricos y de control sobre las cuatro variables de gestión: costo, calendario, calidad y funcionalidad.

3.4.1 Monitoreo de las Variables de Gestión

- **Calendario (Tiempo):** Es la variable más crítica. Durante la Fase 1, se controla mediante el cumplimiento de las fechas límites de los entregables documentales. Durante la Fase 2, el seguimiento diario y operativo se realizará visualmente mediante el **Burndown Chart** de cada Sprint. La pendiente real de "quemado" de horas o puntos de historia se contrastará contra la pendiente ideal para detectar retrasos de forma temprana.
- **Funcionalidad (Alcance):** Se controlará mediante el estado del *Product Backlog* en la herramienta de gestión. Las Historias de Usuario transitarán por estados estrictos (*To Do*, *In Progress*, *Testing*, *Done*). El cumplimiento del alcance se mide basándose en los Criterios de Aceptación definidos para cada Caso de Uso.
- **Calidad:** Se supervisará mediante la ejecución de un plan de pruebas unitarias y funcionales sobre los módulos críticos. Los indicadores clave de calidad incluirán el porcentaje de Casos de Uso validados con éxito y la verificación de la inalterabilidad de los archivos adjuntos (validación de hashes SHA-256 correctos).
- **Costo (Presupuesto):** Al estar acotado a un presupuesto financiero de \$0 USD, el rastreo no se basará en flujos de caja, sino en el consumo de recursos de las

capas gratuitas (*Free Tiers*) de AWS y Supabase. Se configurarán alertas de facturación automáticas en los paneles de control de los proveedores cloud para asegurar que no se incurra en costos excedentes.

3.4.2 Informes: Contenido, Formato, Frecuencia y Estructura

Al tratarse de un desarrollo individual, los informes se unifican en artefactos ágiles y tableros compartidos con la cátedra, estructurados de la siguiente manera:

- **Tablero de Avance Técnico (Tiempo Real):**
 - **Contenido:** Estado de las tareas actuales, Historias de Usuario del Sprint y bloqueos técnicos.
 - **Formato:** Tablero Kanban digitalizado.
 - **Frecuencia:** Actualización diaria por parte del desarrollador.
- **Informe de Cierre de Sprint (Cada 2 o 3 semanas):**
 - **Contenido:** Resumen de velocidad del Sprint, Historias de Usuario completadas (incremento de software), **Burndown Chart consolidado** del ciclo finalizado con el análisis de sus desviaciones, y el estado de la lista de riesgos.
 - **Formato:** Documento ejecutivo breve (PDF) o minuta técnica cargada en el entorno de gestión.
 - **Estructura:** 1. Objetivos del Sprint; 2. Trabajo Entregado vs. Comprometido; 3. Análisis del Burndown Chart; 4. Impedimentos y Riesgos; 5. Plan de Acción.

3.4.3 Mecanismos de Auditoría

El proceso de auditoría es externo y riguroso, coordinado por el cuerpo docente del Seminario de Integración Profesional en su rol de auditores de procesos y calidad de software.

- **Auditorías de Fin de Fase (Hitos de Control):** Revisiones formales y síncronas al cierre de cada cuatrimestre. En la Fase 1, se auditará la consistencia lógica de la arquitectura (DER, ERS y Casos de Uso). En la Fase 2, se realizarán demostraciones funcionales en vivo (*Live Demos*) del software para validar que las prestaciones del MVP coincidan con los requerimientos aprobados.
- **Auditoría de Código y Configuración:** Inspecciones sobre el repositorio de control de versiones (GitHub) para verificar el cumplimiento de buenas prácticas de desarrollo, consistencia en los *commits*, arquitectura limpia en las capas de Flutter/NestJS y la correcta aplicación del control de cambios documental.

3.4.4 Sitio Web y Entorno de Gestión del Proyecto

El proyecto no dispondrá de un sitio web público de marketing, sino de un **Entorno Digital de Gestión y Trazabilidad** restringido para el desarrollador y los evaluadores de la cátedra. Este espacio centralizado se compone de:

- **Repositorio Centralizado (GitHub):** Aloja el código fuente, los scripts de la base de datos PostgreSQL y la documentación técnica versionada (como este SDP y la ERS).
- **Espacio de Gestión Ágil (Jira / GitHub Projects):** Actúa como el centro operativo del proyecto donde reside el *Product Backlog*, las planificaciones de los Sprints y los gráficos automatizados de *Burndown*.
- **Repositorio de Artefactos de Diseño (Figma):** Enlace interactivo permanente a los esquemas de UI/UX y mockups de alta fidelidad, asegurando la trazabilidad entre las pantallas construidas y el diseño aprobado.

3.4.5 Contabilidad del Tiempo

Para medir el esfuerzo real invertido y calibrar la velocidad de desarrollo en las estimaciones futuras, se implementará un registro estricto del tiempo:

- **Mecanismo:** Cada tarea técnica o actividad de gestión creada en el backlog tendrá una estimación inicial en horas de esfuerzo.
- **Registro:** El desarrollador contabilizará y cargará de forma diaria las horas reales dedicadas a cada tarea mediante herramientas de *time-tracking* (ej. Clockify o el sistema nativo de Jira).
- **Análisis:** Al cierre de cada Sprint, la diferencia entre las horas estimadas y las horas reales se utilizará en la *Sprint Retrospective* para ajustar el factor de productividad personal, garantizando que los compromisos de los Sprints subsiguientes sean cada vez más realistas y predecibles.

3.5 Plan de Dotación del Personal

El proyecto ConstructING será desarrollado de manera individual, por lo que todas las actividades de gestión, análisis, diseño, desarrollo, pruebas y documentación estarán a cargo de Nicolás del Río, quien asume la totalidad de los roles definidos en la sección 2.4. Cabe destacar que la participación de dichos roles variará según la etapa del desarrollo y las actividades planificadas para cada fase del proyecto. El responsable del proyecto participará durante todas las etapas del ciclo de vida del software y deberá contar con conocimientos relacionados con programación frontend y backend, diseño de interfaces de usuario, bases de datos, integración de servicios externos y pruebas de software. Asimismo, será necesario adquirir y profundizar conocimientos sobre determinadas tecnologías y herramientas específicas utilizadas en el proyecto, especialmente aquellas vinculadas a la implementación de la arquitectura offline-first, la criptografía nativa para la generación de firmas digitales y hashes SHA-256, y la georreferenciación de evidencias. La dedicación al proyecto se mantendrá constante durante todas las etapas del desarrollo, ajustándose a la planificación definida para cada sprint, y no habrá incorporación de personal adicional, por lo que tampoco se prevé una eliminación gradual del mismo. Todas las responsabilidades permanecerán centralizadas en el responsable del proyecto durante toda su ejecución.

4. PROCESO TÉCNICO

4.1 Métodos, Herramientas y Técnicas

Para la materialización del ecosistema **ConstructING**, se ha definido un *stack* tecnológico y un conjunto de herramientas modernas que aseguran la mantenibilidad, escalabilidad y calidad del código, ajustadas a la realidad de un desarrollo unipersonal.

4.1.1 Entorno del Sistema Informático (Hardware y SO)

- **Entorno de Desarrollo:** El desarrollo se ejecutará en una estación de trabajo local (Windows) con recursos suficientes para virtualizar dispositivos (Emuladores Android) y levantar contenedores locales (Docker) para simular la base de datos.
- **Entorno de Producción (Servidor):** El backend y la base de datos PostgreSQL se alojarán en un entorno *Cloud* basado en distribuciones Linux (ej. Ubuntu Server en AWS o contenedores gestionados en Supabase).
- **Entorno Cliente (Dispositivos Móviles):** La aplicación móvil final estará compilada en un formato instalable (APK) optimizado para el sistema operativo Android (versión 8.0 o superior), considerando especificaciones de hardware estándar (cámara y GPS integrados).

4.1.2 Herramientas de Software

Para maximizar la eficiencia en el desarrollo unipersonal, se unifica el entorno tecnológico mediante las siguientes herramientas automatizadas:

- **Entorno de Desarrollo Integrado (IDE):** Visual Studio Code y Android Studio, equipados con las extensiones oficiales para el análisis estático de Dart y TypeScript.
- **Diseño y Prototipado:** Figma para el modelado de la interfaz de usuario (UI), esquemas de interacción (UX) y exportación de componentes visuales.
- **Control de Código Fuente:** Git como motor local y GitHub como repositorio centralizado en la nube para el resguardo e historial de cambios.
- **Seguimiento de Defectos y Gestión:** Jira / GitHub Projects para la administración del Product Backlog, asignación de Historias de Usuario y control del *Burndown Chart*.
- **Contabilidad del Tiempo:** Clockify / Jira Time Tracking para auditar de forma exacta las horas de esfuerzo invertidas por tarea.

4.1.3 Metodologías, Lenguajes y Estándares de Codificación

El ecosistema se regirá bajo los siguientes lineamientos de ingeniería:

- **Lenguajes de Programación:** *Dart* (versión 3.x o superior) para el desarrollo del cliente móvil por su robustez en la compilación nativa y tipado fuerte, y *TypeScript*

para el backend NestJS para garantizar homogeneidad, tipado estricto y prevención de errores en tiempo de diseño.

- **Estándares de Codificación (Linting):** Se adoptarán las reglas estrictas de `flutter_lints` para el código Dart, y las directrices de `ESLint` combinadas con `Prettier` para NestJS. Ningún incremento de código será integrado a la rama principal si presenta advertencias (*warnings*) de formato o de tipado.
- **Estrategia de Ramas (Branching Workflow):** Se implementará una adaptación simplificada de GitFlow consistente en una rama estable (`main`) y ramas de características de corta duración (`feature/ID_CasoDeUso`). Las integraciones se realizarán mediante Pull Requests auto-evaluadas y verificadas localmente.

4.1.4 Técnicas de Verificación y Validación

Las prácticas destinadas a asegurar que el software se construye correctamente y cumple con los requerimientos técnicos constan de:

- **Pruebas Unitarias:** Validación de la lógica de negocio pura (cálculo de coordenadas, validación de estados de hitos, formato de strings) aislada de la infraestructura, utilizando `flutter_test` (Dart) y `Jest` (NestJS).
- **Depuración Automática y Análisis de Código:** Uso de herramientas de depuración nativas de los IDEs para el seguimiento de variables en tiempo de ejecución, control de fugas de memoria (*memory leaks*) en el renderizado de Flutter y perfiles de rendimiento en las consultas PostgreSQL.

4.2 Documentación del Software

El ecosistema **ConstructING** requiere una base documental sólida para asegurar la trazabilidad entre los requerimientos del negocio y el código implementado. Al tratarse de un proyecto individual bajo supervisión académica, la documentación servirá como principal artefacto de auditoría.

Listado de documentos:

Nombre Documento	del	Etapas de Desarrollo	de	Instancias de Revisión	de	Criterios de Aceptación (Firma)	de

Project Proposal (Visión y Alcance)	Fase 1 - Iniciación	Revisión inicial de propuesta.	Aprobación por parte de la cátedra para avanzar.
Plan de Desarrollo de Software (SDP - IEEE 1058)	Fase 1 y 2 (Documento Vivo)	Revisiones periódicas al finalizar cada hito.	Aprobación de la cátedra; línea base del cronograma fijada.
Matriz de Riesgos (Top 10)	Fase 1 (Actualizado en Fase 2)	En cada <i>Sprint Planning</i> / <i>Retrospective</i> .	Aceptación interna del desarrollador (Risk Manager).
Especificación de Requisitos (ERS - IEEE 830)	Fase 1 - Elaboración	Revisión de completitud y ambigüedad.	Línea base firmada por la cátedra antes de codificar.
Especificación de Casos de Uso (Anexo ERS)	Fase 1 - Elaboración	Revisión técnica junto a la ERS.	Trazabilidad del 100% contra los Requerimientos Funcionales.
Arquitectura de Software y Datos (DER / Relacional)	Fase 1 - Diseño	Inspección de normalización (PostgreSQL) y restricciones.	Aprobación del Arquitecto (Autor) y Cátedra.

Guía de Estilo y UI Mockups (Figma)	Fase 1 - Diseño	Revisión de usabilidad.	Prototipo de alta fidelidad validado visualmente.
Artefactos de Scrum (Backlog y Burndown Charts)	Fase 2 - Construcción	<i>Sprint Reviews</i> (Cada 2/3 semanas).	Aceptación empírica demostrando el software funcionando.
Documentación de API REST (Swagger/OpenAPI)	Fase 2 - Construcción	Revisión continua mediante analizadores estáticos.	Cobertura total de los <i>endpoints</i> consumidos por Flutter.
Manual de Despliegue (AWS / Supabase)	Fase 2 - Transición	Previo a la defensa final.	Despliegue exitoso del MVP en la nube.
Presentación y Balance Final del Proyecto	Fase 2 - Cierre	Día de la Defensa (Diciembre 2026).	Aprobación final y calificación por parte del cuerpo docente.

4.3 Funciones Soporte del Proyecto

Para garantizar el éxito del ecosistema **ConstructING** y el cumplimiento de los estándares de la cátedra, se han definido planes específicos para las funciones de soporte que acompañan al desarrollo del código. Al tratarse de un proyecto unipersonal, estas funciones recaen sobre el único desarrollador, pero se estructuran metodológicamente como procesos formales.

1. Gestión de la Configuración del Software (SCM)

Este plan asegura que todos los artefactos del proyecto (código, documentos, modelos) mantengan integridad, trazabilidad y control de versiones a lo largo del tiempo.

- **Recursos y Herramientas:** Se utilizará **Git** de forma local y **GitHub** (repositorio privado/público gratuito) como servidor central. Para la gestión de entornos, se utilizarán archivos .env excluidos del repositorio para proteger credenciales de bases de datos y claves API.
- **Responsabilidades:** El desarrollador es responsable de no inyectar código directo a la rama de producción. Se aplicará un flujo de trabajo simplificado basado en Feature Branches (una rama por cada Historia de Usuario) y Pull Requests auto-revisados.
- **Calendario:** Proceso continuo y obligatorio durante toda la Fase 2 (Construcción del MVP).
- **Presupuesto:** \$0 USD (Nivel gratuito de GitHub y herramientas Open Source).

2. Aseguramiento de la Calidad (QA - Quality Assurance)

El plan de QA está diseñado para mitigar los riesgos tecnológicos críticos identificados en la sección 3.3, específicamente la inalterabilidad de la firma digital y la sincronización offline-first.

- **Recursos y Herramientas:** Frameworks de pruebas unitarias (flutter_test para Dart, jest para NestJS). Pruebas de integración de API mediante **Postman**. Simuladores de dispositivos móviles (Android Emulator) para pruebas de caja negra.
- **Responsabilidades:** El desarrollador asume el rol de QA Analyst en la última etapa de cada Sprint. Ninguna Historia de Usuario podrá pasar a la columna de Done (Completado) sin antes haber superado sus Criterios de Aceptación.
- **Procedimientos Críticos:** Se realizarán pruebas de estrés manuales simulando pérdida de conectividad ("Modo Avión") para forzar la persistencia en la base de datos local y evaluar la correcta sincronización de evidencias al recuperar la conexión. Se validarán matemáticamente los hashes SHA-256 generados en los reportes PDF.
- **Calendario:** Integrado iterativamente al cierre de cada Sprint (Agosto - Noviembre 2026).
- **Presupuesto:** \$0 USD.

3. Documentación para el Usuario Final

Dado que el propósito de **ConstructING** es reducir la asimetría de información y facilitar la gestión a Propietarios y Profesionales, la adopción del sistema no debe suponer una barrera técnica.

- **Recursos y Entregables:** En lugar de un manual físico extenso, se desarrollará un **Módulo de Onboarding Interactivo** nativo dentro de la aplicación móvil (pantallas de bienvenida y tooltips instructivos en las vistas principales). Adicionalmente, se entregará una breve Guía de Usuario en formato PDF (desplegada en la nube) que detalle cómo validar la autenticidad de las actas certificadas.
- **Responsabilidades:** El desarrollador es responsable de la redacción técnica y el diseño instruccional del material.
- **Calendario:** Se desarrollará durante la Etapa 4 (Despliegue y Transición) en Noviembre de 2026, una vez que el MVP y las interfaces (UI) estén estabilizadas y no sufran alteraciones.
- **Presupuesto:** \$0 USD.

5. PAQUETES DE TRABAJO, CALENDARIO Y PRESUPUESTO.

5.1 Paquetes de Trabajo

5.1 Paquetes de Trabajo (EDT por Módulos Funcionales)

Para organizar y controlar el esfuerzo de desarrollo del ecosistema ConstructING de manera sistemática y trazable, el trabajo de construcción del software se ha dividido en Paquetes de Trabajo (PT). Cada PT corresponde a un módulo funcional de la arquitectura del sistema y agrupa los Casos de Uso (CdU) especificados en la ERS que deben ser desarrollados, probados e integrados para dar por finalizado ese componente.

A continuación, se detalla la tabla de los paquetes de trabajo del desarrollo del MVP (Fase 2):

ID del Paquete	Nombre del Paquete de Trabajo (Módulo)	Casos de Uso (CdU) Asociados

PT-01	Gestión de Identidad y Autenticación	CU-01: Iniciar Sesión CU-02: Validar Credenciales CU-03: Recuperar Contraseña CU-04: Cerrar Sesión CU-05: Gestionar Sesión Persistente CU-06: Crear perfil de usuario CU-07: Modificar datos de usuario
-------	--------------------------------------	---

		CU-10: Validar unicidad de identidad CU-11: Encriptar credenciales de acceso CU-12: Enviar correo de bienvenida automática
PT-02	Administración de Usuarios y Roles (RBAC)	CU-08: Dar de baja / Inhabilitar usuario CU-09: Consultar listado general de usuarios

PT-03	Gestión de Obras (Proyectos)	CU-13: Crear Nueva Obra CU-14: Vincular Propietario a Obra CU-15: Establecer Ubicación Geográfica de la obra CU-16: Validar Datos Obligatorios de obra CU-17: Modificar Información de Obra CU-18: Consultar Obras Asignadas CU-19: Visualizar Ficha Técnica de Obra
--------------	-------------------------------------	---

		CU-20: Actualizar Estado del Proyecto CU-21: Archivar Obra CU-22: Generar Código de Invitación
--	--	---

PT-04	Planificación y Hoja de Ruta (Hitos)	CU-23: Crear Hito de Obra CU-24: Establecer Dependencias CU-25: Modificar Detalle de Hito CU-26: Actualizar Estado de Hito CU-27: Eliminar Hito CU-28: Validar Progresión de Estado CU-29: Calcular Ruta Crítica
--------------	---	---

		CU-30: Recalcular Duración Total del proyecto
--	--	---

PT-05	Registro de Georreferenciada	Evidencia	CU-31: Inicializar Interfaz de Captura CU-32: Capturar Fotografía de Avance CU-33: Grabar video corto de evidencia CU-34: Obtener Ubicación Automática CU-35: Validar Cercanía al punto de Obra CU-36: Estampar Marca de agua Inalterable CU-37: Bloquear Acceso a Galería
-------	------------------------------	-----------	---

		CU-38: Validar Límite de tamaño y duración CU-39: Redactar observación técnica CU-40: Visualizar Mapa de Relevamiento CU-41: Descartar evidencia no guardada
--	--	--

PT-06	Sincronización y Tolerancia a Fallos (Offline-First)	CU-42: Almacenar Datos en Caché Local CU-43: Monitorear Estado de Conectividad CU-44: Iniciar Proceso de Sincronización a la Nube CU-45: Validar Integridad de Archivos Sincronizados CU-46: Reanudar Carga Interrumpida CU-47: Resolver Conflictos Temporales de Datos
--------------	---	--

		CU-48: Consultar Estado de Sincronización CU-49: Forzar Sincronización Manual
--	--	---

PT-07	Certificación Digital y Firma en Pantalla	CU-50: Solicitar Certificación de Hito CU-51: Visualizar resumen de evidencias a certificar CU-52: Registrar Firma Manuscrita en pantalla CU-53: Limpiar / Reintentar Trazo de Firma CU-54: Descargar Acta de Conformidad CU-55: Capturar Metadatos de Trazo CU-56: Generar Documento PDF
--------------	--	--

		CU-57: Bloquear Registros Editables
PT-08	Seguridad Criptográfica y Trazabilidad (Audit Logs)	CU-58: Calcular Hash SHA-256 CU-59: Generar Hash de Acta PDF CU-60: Registrar Evento de Auditoría

		CU-61: Verificar Integridad Documental CU-62: Disparar Alerta por Discrepancia de Integridad CU-63: Consultar Reporte de Auditoría
PT-09	Línea de Tiempo y Visualización de Avances	CU-64: Visualizar Línea de Tiempo CU-65: Acceder a Detalle Técnico CU-66: Consultar Dashboard de Obras

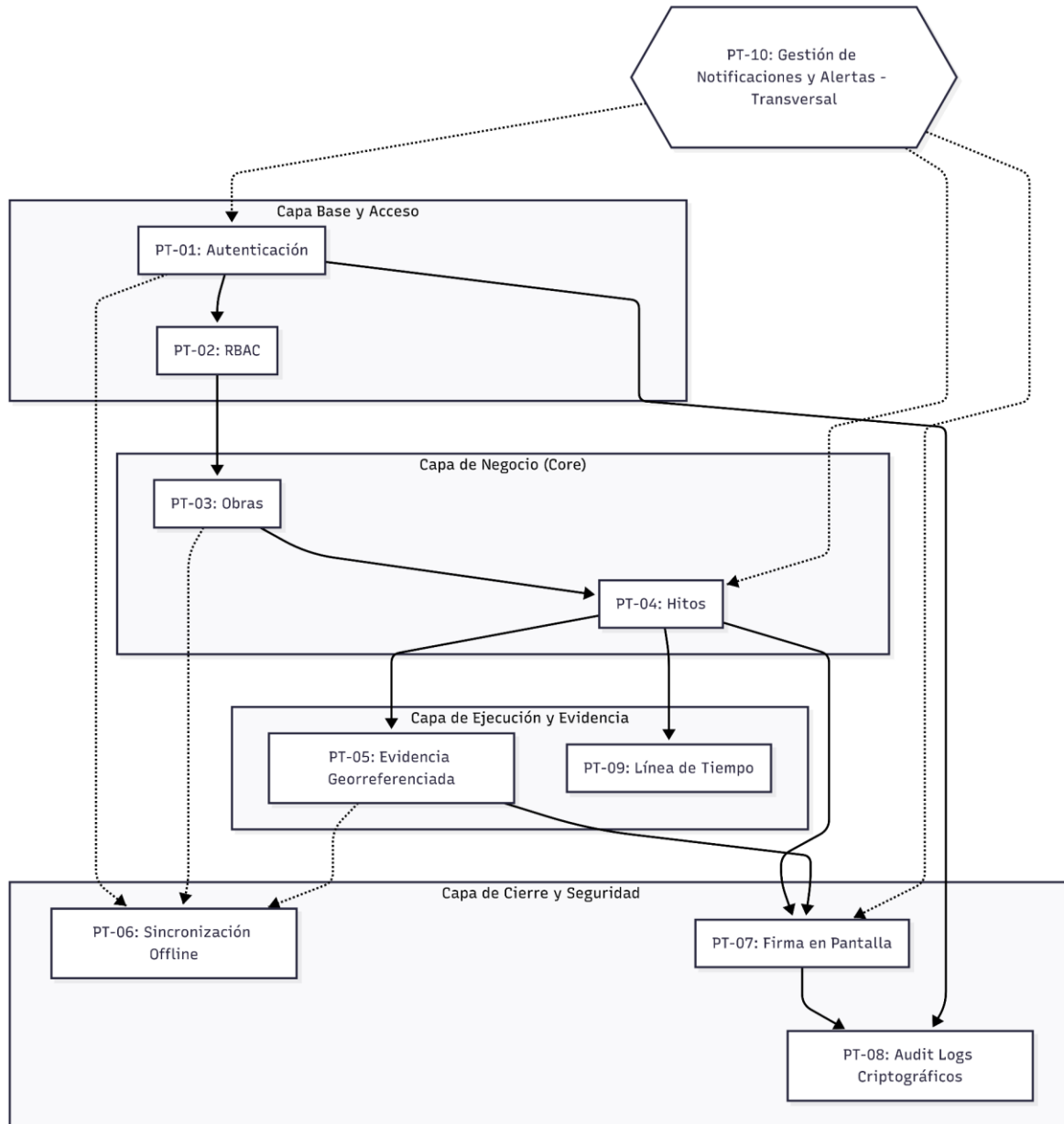
PT-10	Gestión de Notificaciones y Alertas	CU-67: Generar Notificación de Avance CU-68: Notificar Solicitud de Firma CU-69: Enviar Alerta de Retraso CU-70: Visualizar Historial de notificaciones CU-71: Configurar Preferencia de Alerta CU-72: Marcar Notificación Leída
--------------	--	--

5.2 Dependencias

Para garantizar una ejecución eficiente y mitigar riesgos de reprocesamiento durante la Fase 2 (Desarrollo del MVP), se han determinado las dependencias críticas del proyecto. Estas se dividen en dependencias internas (relaciones de precedencia entre los paquetes

de trabajo de software) y dependencias externas (vínculos con factores e hitos ajenos al control directo del desarrollo).

5.2.1 Mapa de Precedencia Temporal y Funcional



5.2.2 Dependencias Internas (Entre Paquetes de Trabajo)

La secuenciación del desarrollo se basa en dependencias de tipo **Fin-Inicio (FI)**, lo que significa que un paquete de trabajo predecesor debe estar completamente funcional (o disponer de sus interfaces/APIs estables) antes de iniciar la construcción del paquete sucesor.

A continuación, se define la matriz de relaciones y el tipo de impacto arquitectónico:

ID Paquete	Nombre del Paquete de Trabajo	Paquetes Predecesores Críticos	Justificación Técnica y Arquitectónica
PT-01	Gestión de Identidad y Autenticación	Ninguno	Es la base transversal del sistema. Provee el Contexto de Usuario y los tokens de seguridad necesarios para cualquier petición al backend.
PT-02	Administración de Usuarios y Roles (RBAC)	PT-01	Depende de la existencia de entidades de usuario autenticadas para poder asociar, validar e inhabilitar los roles (<i>Profesional, Propietario, Admin</i>).
PT-03	Gestión de Obras (Proyectos)	PT-02	Las obras no pueden existir de forma aislada; requieren el módulo RBAC para vincular un <i>Profesional</i> responsable y un <i>Propietario</i> al registro del proyecto.

PT-04	Planificación y Hoja de Ruta (Hitos)	PT-03	Estructura la descomposición técnica de los proyectos. Un hito obligatoriamente depende de la existencia previa de una obra activa a la cual pertenecer.
PT-05	Registro de Evidencia Georreferenciada	PT-04	Las fotografías, videos y observaciones técnicas se capturan y guardan indexadas bajo un hito específico. Sin la estructura de hitos, la evidencia carece de anclaje relacional.
PT-06	Sincronización Offline-First	PT-01, PT-03, PT-05	Actúa como capa intermedia de persistencia. Necesita que las estructuras de datos de usuarios, obras y evidencias estén consolidadas localmente en SQLite/Hive para gestionar el pipeline de sincronización y la cola de peticiones hacia la nube.

PT-07	Certificación Digital y Firma en Pantalla	PT-04, PT-05	El acta de conformidad digital (PDF) es una consolidación legal. Requiere que el hito esté completado (PT-04) y que existan evidencias visuales inalterables registradas (PT-05) para conformar el documento que se procederá a firmar.
PT-08	Seguridad Criptográfica y Audit Logs	PT-01, PT-07	Para calcular el hash inalterable SHA-256 del acta, el PDF debe haber sido generado (PT-07). Asimismo, requiere el estado de autenticación (PT-01) para estampar la IP, ID de usuario y metadatos en el log de auditoría inmutable.
PT-09	Línea de Tiempo y Visualización	PT-03, PT-04, PT-05	Es un módulo de consumo visual de datos. Depende de que las obras (PT-03), los hitos (PT-04) y las evidencias (PT-05) existan y estén expuestas mediante servicios para poder renderizar la interfaz cronológica sin errores de nulidad.

PT-10	Gestión de Notificaciones y Alertas	Todos los PTs	Funciona mediante un patrón de arquitectura orientada a eventos (Event-Driven). Depende de los triggers disparados por los otros módulos (ej: nueva obra en PT-03, retraso en PT-04, solicitud de firma en PT-07).
--------------	-------------------------------------	---------------	--

5.2.3 Dependencias Externas

El éxito del proyecto y el cumplimiento del cronograma global de la materia están condicionados por los siguientes eventos y servicios externos:

- **Disponibilidad de Servicios de Geolocalización (APIs de Terceros):** La capacidad de georreferenciación inalterable del **PT-05** depende de la disponibilidad operativa de los servicios de ubicación del sistema operativo del dispositivo móvil (Google Location Services en Android / Apple Core Location en iOS) y de las cuotas de servicios de mapas (Google Maps API o Mapbox). Una falla o cambio de políticas en estas APIs bloquearía la validación de cercanía a la obra.
- **Provisión de la Infraestructura de Servidores y Cloud (BaaS):** El pipeline de sincronización del **PT-06** y el almacenamiento de actas/multimedia dependen de la correcta configuración y disponibilidad del Backend-as-a-Service seleccionado (ej. Supabase / AWS). Demoras en el aprovisionamiento de estos entornos actuarán como un cuello de botella para las pruebas integradas.
- **Proceso de Aprobación en Tiendas de Aplicaciones (Play Store / App Store):** Aunque para los fines del seminario de integración el MVP puede distribuirse mediante APKs directas o entornos de prueba (TestFlight / Firebase App Distribution), cualquier intención de despliegue formal o simulación final de entorno real queda supeditada a los tiempos de revisión técnicos y burocráticos impuestos por Google y Apple.
- **Validación Legal y Normativa de la Firma Manuscrita en Pantalla:** La validez y el alcance de las actas generadas en el **PT-07** dependen de los marcos regulatorios locales vigentes sobre firma electrónica (Ley 25.506 en Argentina). Cualquier cambio normativo drástico sobre la validez de firmas biométricas en pantallas táctiles podría obligar a rediseñar las restricciones de validación del software.

5.3 Recursos Necesitados

Al tratarse de un proyecto de carácter unipersonal y académico, la gestión de los recursos se enfoca en la optimización del tiempo del único desarrollador, el uso de infraestructura de desarrollo local y el aprovechamiento de herramientas bajo licenciamiento gratuito o de código abierto (*Open Source*).

5.3.1 Clasificación de Recursos Globales

- **Recursos Humanos (Personal):** Un (1) único integrante que asume los roles combinados de Gerente de Proyecto, Analista de Requisitos, Arquitecto de Software, Desarrollador (Fullstack) y Analista de Calidad (QA), con una asignación del 100% de dedicación durante el ciclo lectivo.
- **Recursos de Hardware:**
 - Una (1) estación de trabajo (PC) con capacidades de virtualización para la ejecución de entornos integrados y bases de datos locales.
 - Un (1) dispositivo móvil físico con Sistema Operativo Android (versión 8.0 o superior) para pruebas de rendimiento en caliente y simulación de pérdida de conectividad.
- **Recursos de Software:**
 - *Diseño y Gestión:* Figma (Plan educativo/gratuito), Jira Software o GitHub Projects, y Microsoft Word para el mantenimiento de la documentación base (PdS, ERS, SDP).
 - *Desarrollo e Integración:* Visual Studio Code / Android Studio, Git (Local), GitHub (Repositorio remoto), Postman (Pruebas de API) y Docker (Opcional para entornos locales).
 - *Infraestructura Cloud:* Capas gratuitas (*Free Tiers*) de Supabase (PostgreSQL y autenticación) y AWS.
- **Material de Oficina y Logística:** Insumos digitales, papelería mínima para anotaciones de diseño de algoritmos, y conectividad a internet residencial estándar. No se requieren espacios físicos corporativos ni oficinas dedicadas.
- **Entrenamiento y Capacitación:** Esfuerzo invertido en auto-capacitación técnica sin costo financiero. Se centra en la investigación de librerías de persistencia local en Flutter para el motor *Offline-First* (Hive o Isar) y en la implementación de criptografía nativa para la generación de firmas digitales y hashes SHA-256.

5.3.2 Evolución y Asignación de Recursos Mes a Mes (Ciclo 2026)

La necesidad y el uso de los recursos varían sustancialmente según la fase académica del proyecto, distribuyéndose a lo largo del año de la siguiente manera:

Período (Meses)	Fase / Paquetes EDT	Person al Activo	Hardwar e Requerid o	Software Clave Utilizado	Enfoque de Entrenamien to
Marzo - Abril	Iniciación y Requisitos (Fase 1)	1	1 PC de desarroll o	MS Word, Navegador Web, MS Excel	Elicitación y estándares de especificació n (IEEE 830).
Mayo - Junio	Ingeniería y Diseño (Fase 1)	1	1 PC de desarroll o	Figma, Draw.io, MS Word,MS Excel	Modelado de datos relacionales y diseño de interfaces móviles (UI/UX).
Julio	Cierre de Fase 1 (Fase 1)	1	1 PC de desarroll o	MS Word, Figma, GitHub	Consolidación de la tríada documental (PdS, ERS, SDP).
Agosto - Septiemb re	Construcció n MVP - Núcleo (PT- 01, PT-02, PT-03, PT- 04)	1	1 PC + 1 Celular Android	VS Code, Flutter, NestJS, Supabase Cloud	Arquitectura Limpia en Flutter y construcción de APIs

					robustas en NestJS.
Octubre	Construcción MVP - Seguridad (PT-05, PT-07, PT-08)	1	1 PC + 1 Celular Android	VS Code, Flutter, NestJS, Postman	Criptografía aplicada: Algoritmos de Hash SHA-256 e inalterabilidad de PDFs.
Noviembre	Pruebas de Calidad y Estabilización (PT-06, PT-09, PT-10)	1	1 PC + 1 Celular Android	Flutter Test, Jest, Postman, Clockify	<i>Testing</i> de estrés en modo <i>offline</i> y automatización de pruebas unitarias.
Diciembre	Despliegue y Defensa Final	1	1 PC + 1 Celular Android	Android SDK (Generación APK), MS PowerPoint	Técnicas de oratoria, preparación de <i>Live Demo</i> y empaquetado de software.

5.4 Asignación del Presupuesto y de los Recursos

El ecosistema ConstructING cuenta con un presupuesto financiero asignado de \$0 USD, debido a que todas las soluciones tecnológicas elegidas (Flutter, NestJS, PostgreSQL) son de código abierto y su despliegue se realiza bajo las capas gratuitas (*Free Tiers*) de proveedores cloud como Supabase y AWS. Por lo tanto, la contabilidad presupuestaria del proyecto no se mide en flujos de caja tradicionales, sino en un **presupuesto de esfuerzo (Horas Ideales de Trabajo)** invertidas en su totalidad por el único integrante del equipo de ingeniería.

5.4.1 Distribución Presupuestaria por Función de Ingeniería

El presupuesto total de esfuerzo estimado para el ciclo de vida completo del proyecto es de **400 horas**. Este recurso temporal se distribuye estratégicamente entre las diferentes disciplinas funcionales de la ingeniería de software para asegurar un equilibrio óptimo entre la calidad documental, el diseño de arquitectura, la codificación y las pruebas de regresión:

- **Función de Gestión y Control (12.5% - 50 horas):** Comprende el seguimiento metodológico, la actualización sistemática del Product Backlog, el análisis y mitigación continua de la matriz de riesgos, y la confección evolutiva de este Plan de Desarrollo de Software (SDP) (Ref: WBS 1.1).
- **Función de Ingeniería de Requisitos (15.0% - 60 horas):** Dedicado exclusivamente a las tareas de elicitación de necesidades con expertos del dominio, especificación de la ERS bajo el estándar IEEE 830 y el modelado formal de los escenarios de interacción (Ref: WBS 1.2).
- **Función de Diseño Arquitectónico (22.5% - 90 horas):** Inversión de esfuerzo combinada en el modelado de datos relacionales (DER PostgreSQL, tablas e índices), la especificación de lógica local y el prototipado de interfaces UI/UX interactivo de alta fidelidad en Figma (Ref: WBS 1.3 y 1.4).
- **Función de Construcción Técnica (37.5% - 150 horas):** El núcleo operativo del proyecto. Contempla la programación pura de la lógica de negocio backend (NestJS) y mobile (Flutter), la persistencia nativa para soporte offline y el desarrollo granular del motor criptográfico basado en la estimación *Bottom-Up* por Caso de Uso (Ref: WBS 2.0 integral).
- **Función de Aseguramiento de la Calidad - QA (5.0% - 20 horas):** Esfuerzo reservado específicamente para el diseño y ejecución de pruebas unitarias, validación de la integración de APIs, simulaciones de conflictos de datos en redes inestables y pruebas integrales de estrés en escenarios sin conectividad (Ref: WBS 3.1).
- **Función de Despliegue y Cierre Académico (7.5% - 30 horas):** Consumo final de tiempo orientado al aprovisionamiento de infraestructura cloud (Supabase), compilación y empaquetado del binario ejecutable final (APK Android), redacción de manuales técnicos y la preparación de la Demo en vivo para la defensa (Ref: WBS 3.2, 3.3 y 3.4).

5.4.2 Curva de Consumo Presupuestario (Evolución Temporal del Gasto)

Al igual que en un entorno corporativo donde el capital financiero se agota mes a mes, el "gasto" de las horas de esfuerzo se distribuye siguiendo el ciclo de vida del software establecido en el calendario (Sección 5.5). La tasa de consumo acumulada responde estrictamente a la siguiente estructura cuatrimestral:

- **Primer Cuatrimestre (Marzo – Julio): Consumo del 50.0% del Presupuesto (200 horas).** Durante este período se devengan las horas asignadas en su totalidad a las funciones de Gestión, Ingeniería de Requisitos y Diseño Arquitectónico (WBS 1.0). Al cierre de este hito lectivo, la línea base documental y los prototipos de Figma quedan completamente congelados.
- **Segundo Cuatrimestre (Agosto – Noviembre): Consumo del 37.5% del Presupuesto (150 horas).** Representa el pico máximo de gasto operativo y de codificación. El esfuerzo se consume a un ritmo constante de **25 horas por Sprint** (a lo largo de los 6 Sprints definidos), enfocándose de manera exclusiva en la materialización y testeo unitario de los 72 Casos de Uso del sistema (WBS 2.0).
- **Cierre del Proyecto (Diciembre): Consumo del 12.5% del Presupuesto (50 horas).** Se agotan las últimas unidades del presupuesto en las funciones críticas de Aseguramiento de la Calidad (QA integral), Despliegue en producción en la nube, compilación de entregables finales y defensa académica del MVP (WBS 3.0).

5.5 Calendario

El cronograma operativo del proyecto ConstructING se estructura mediante hitos temporales fijos (fechas absolutas definidas por el ciclo lectivo 2026 de la USAL) y plazos lógicos de ejecución (fechas relativas). Esta combinación garantiza la flexibilidad metodológica necesaria para absorber desvíos sin comprometer la entrega final.

5.5.1 Fechas Relativas de los Hitos de Control

Tomando como base las dependencias de la sección 5.2, el progreso documental y de gestión se rige por las siguientes distancias de tiempo:

- **Línea Base del Alcance (ERS):** Fecha de Inicio del Proyecto + 60 días nominales.
- **Línea Base del Diseño Técnico (PostgreSQL / Figma):** Aprobación de la ERS + 45 días nominales.
- **Lanzamiento del Desarrollo Operativo:** Cierre de Fase 1 + 7 días (Inicio del Sprint 1).
- **Estabilización de Código (Code Freeze):** Inicio de la Fase de Construcción + 90 días (Cierre del Sprint 6).
- **Despliegue y Pruebas Finales (QA):** Code Freeze + 15 días.

5.5.2 Cronograma de Fechas Absolutas por Paquete de Trabajo (Ciclo 2026)

La ejecución temporal de los paquetes definidos en la EDT (Sección 5.1) se distribuye a lo largo del año académico según el siguiente cronograma de hitos:

- **Fase 1: Ingeniería de Requisitos, Diseño y Gestión (Marzo 2026 – Julio 2026)**
 - **Hito de Cierre Documental (Congelamiento de Línea Base):** Mediados de Julio 2026. Entrega final de la ERS v0.8.0, Modelos de Datos y Prototipos en Figma.
- **Fase 2: Construcción Iterativa del MVP - 2do Cuatrimestre (Agosto 2026 – Noviembre 2026)**
 - **Hito de Lanzamiento del Desarrollo Operativo:** Primera semana de Agosto 2026 (Inicio de la codificación activa tras el receso académico).
 - **Sprint 1 (Agosto - Semanas 1 a 3):** Configuración de entornos y desarrollo de **PT-01** (Autenticación) y **PT-02** (RBAC).
 - **Sprint 2 (Agosto - Semana 4 a Septiembre - Semana 2):** Desarrollo de **PT-03** (Obras) y **PT-04** (Planificación e Hitos).
 - **Sprint 3 (Septiembre - Semana 3 a Octubre - Semana 1):** Implementación de la captura multimedia en la App Mobile con **PT-05** (Evidencia Georreferenciada).
 - **Sprint 4 (Octubre - Semanas 2 a 4):** Lógica local y consumo visual mediante **PT-06** (Sincronización Offline-First) y **PT-09** (Línea de Tiempo).
 - **Sprint 5 (Noviembre - Semanas 1 a 2):** Núcleo legal criptográfico con **PT-07** (Firma en Pantalla) y **PT-08** (Audit Logs SHA-256).
 - **Sprint 6 (Noviembre - Semanas 3 a 4):** Finalización de servicios con **PT-10** (Notificaciones) y estabilización integral de la App (Code Freeze).
- **Fase 3: Despliegue, Transición y Cierre (Diciembre 2026)**
 - **Hito de Defensa Final:** Primera quincena de Diciembre 2026. Migración a producción (Supabase/AWS), compilación de la APK final y Demo interactiva ante el tribunal.

5.6 Estimación de esfuerzo

Dado que el desarrollo del ecosistema ConstructING se aborda como un proyecto final de carrera unipersonal, la estimación del esfuerzo se realiza mediante unidades funcionales basadas en la Especificación de Requisitos de Software (ERS). Para garantizar la viabilidad del proyecto dentro del año académico 2026, se ha establecido una capacidad de dedicación sostenida de 10 horas semanales durante 40 semanas, resultando en un presupuesto inamovible de **400 horas totales**.

A continuación, se detalla la Estructura de Descomposición del Trabajo (EDT / WBS). La Fase 2 (Construcción) aplica una estimación *Bottom-Up*, desglosando las horas ideales requeridas para la codificación, pruebas unitarias y empaquetado de cada Caso de Uso (CdU) individual, asegurando la trazabilidad absoluta del proceso:

Código WBS	Componente / Paquete de Trabajo / Caso de Uso (CdU)	Esfuerzo Estimado
1.0	Fase 1: Ingeniería, Arquitectura y Gestión (Línea Base Documental)	200 hs
1.1	Gestión de Proyecto, Planificación y Documentación (SDP)	50 hs
1.2	Elicitación y Especificación de Requisitos (ERS y Casos de Uso)	60 hs
1.3	Diseño de Arquitectura de Datos (Modelos y DER PostgreSQL)	50 hs
1.4	Diseño de Interfaces y Prototipado UX/UI de Alta Fidelidad (Figma)	40 hs
2.0	Fase 2: Construcción Iterativa del MVP (Desarrollo por CdU)	150 hs
2.1	Sprint 1: Gestión de Identidad, Autenticación y RBAC (PT-01 / PT-02)	25 hs
2.1.1	CU-01: Iniciar Sesión	2.0 hs

2.1.2	CU-02: Validar Credenciales (Lógica de tokens JWT en Backend)	3.0 hs
2.1.3	CU-03: Recuperar Contraseña	2.0 hs
2.1.4	CU-04: Cerrar Sesión	1.0 hs
2.1.5	CU-05: Gestionar Sesión Persistente (Secure Storage Local)	2.0 hs
2.1.6	CU-06: Crear perfil de usuario	2.0 hs
2.1.7	CU-07: Modificar datos de usuario	2.0 hs
2.1.8	CU-10: Validar unicidad de identidad	2.0 hs
2.1.9	CU-11: Encriptar credenciales de acceso	2.0 hs
2.1.10	CU-12: Enviar correo de bienvenida automática	1.0 hs
2.1.11	CU-08: Dar de baja / Inhabilitar usuario	3.0 hs
2.1.12	CU-09: Consultar listado general de usuarios	3.0 hs

2.2	Sprint 2: Gestión de Obras, Planificación e Hitos (PT-03 / PT-04)	25 hs
2.2.1	CU-13: Crear Nueva Obra	2.0 hs
2.2.2	CU-14: Vincular Propietario a Obra	1.5 hs
2.2.3	CU-15: Establecer Ubicación Geográfica de la obra	2.0 hs
2.2.4	CU-16: Validar Datos Obligatorios de obra	1.0 hs
2.2.5	CU-17: Modificar Información de Obra	1.5 hs
2.2.6	CU-18: Consultar Obras Asignadas	1.5 hs
2.2.7	CU-19: Visualizar Ficha Técnica de Obra	1.5 hs
2.2.8	CU-20: Actualizar Estado del Proyecto	1.0 hs
2.2.9	CU-21: Archivar Obra	1.0 hs
2.2.10	CU-22: Generar Código de Invitación	2.0 hs

2.2.11	CU-23: Crear Hito de Obra	1.5 hs
2.2.12	CU-24: Establecer Dependencias entre Hitos	2.0 hs
2.2.13	CU-25: Modificar Detalle de Hito	1.0 hs
2.2.14	CU-26: Actualizar Estado de Hito	1.0 hs
2.2.15	CU-27: Eliminar Hito	1.0 hs
2.2.16	CU-28: Validar Progresión de Estado	1.0 hs
2.2.17	CU-29: Calcular Ruta Crítica	1.0 hs
2.2.18	CU-30: Recalcular Duración Total del proyecto	0.5 hs
2.3	Sprint 3: Registro de Evidencia Georreferenciada (PT-05)	25 hs
2.3.1	CU-31: Inicializar Interfaz de Captura	1.0 hs
2.3.2	CU-32: Capturar Fotografía de Avance (Integración Plugin de Cámara)	3.0 hs

2.3.3	CU-33: Grabar video corto de evidencia	3.0 hs
2.3.4	CU-34: Obtener Ubicación Automática (API de Geolocalización)	2.0 hs
2.3.5	CU-35: Validar Cercanía al punto de Obra (Geofencing)	3.0 hs
2.3.6	CU-36: Estampar Marca de agua Inalterable (Metadatos/Timestamp)	3.0 hs
2.3.7	CU-37: Bloquear Acceso a Galería (Forzar captura nativa)	2.0 hs
2.3.8	CU-38: Validar Límite de tamaño y duración de archivos	2.0 hs
2.3.9	CU-39: Redactar observación técnica	1.5 hs
2.3.10	CU-40: Visualizar Mapa de Relevamiento	3.0 hs
2.3.11	CU-41: Descartar evidencia no guardada	1.5 hs

2.4	Sprint 4: Arquitectura Offline-First y Línea de Tiempo (PT-06 / PT-09)	25 hs
2.4.1	CU-42: Almacenar Datos en Caché Local (SQLite/Hive en App)	2.5 hs
2.4.2	CU-43: Monitorear Estado de Conectividad (Connectivity Listeners)	2.0 hs
2.4.3	CU-44: Iniciar Proceso de Sincronización a la Nube	3.0 hs
2.4.4	CU-45: Validar Integridad de Archivos Sincronizados	2.0 hs
2.4.5	CU-46: Reanudar Carga Interrumpida de Archivos Pesados	2.0 hs
2.4.6	CU-47: Resolver Conflictos Temporales de Datos	2.5 hs
2.4.7	CU-48: Consultar Estado de Sincronización	1.5 hs
2.4.8	CU-49: Forzar Sincronización Manual	1.5 hs

2.4.9	CU-64: Visualizar Línea de Tiempo Cronológica de la Obra	3.0 hs
2.4.10	CU-65: Acceder a Detalle Técnico del Relevamiento	2.0 hs
2.4.11	CU-66: Consultar Dashboard Consolidado de Obras	3.0 hs
2.5	Sprint 5: Certificación, Firma y Criptografía Legal (PT-07 / PT-08)	25 hs
2.5.1	CU-50: Solicitar Certificación de Hito	1.5 hs
2.5.2	CU-51: Visualizar resumen de evidencias a certificar	2.0 hs
2.5.3	CU-52: Registrar Firma Manuscrita en pantalla (Canvas Component)	3.0 hs
2.5.4	CU-53: Limpiar / Reintentar Trazo de Firma	1.0 hs
2.5.5	CU-54: Descargar Acta de Conformidad	2.0 hs
2.5.6	CU-55: Capturar Metadatos de Trazo Biométrico	2.0 hs

2.5.7	CU-56: Generar Documento PDF (Agrupando actas y firmas en backend)	3.0 hs
2.5.8	CU-57: Bloquear Registros Editables post-firma	1.5 hs
2.5.9	CU-58: Calcular Hash SHA-256 sobre registros	1.5 hs
2.5.10	CU-59: Generar Hash de Acta PDF para inmutabilidad	1.5 hs
2.5.11	CU-60: Registrar Evento de Auditoría (Audit Logs)	2.0 hs
2.5.12	CU-61: Verificar Integridad Documental	1.5 hs
2.5.13	CU-62: Disparar Alerta por Discrepancia de Integridad	1.5 hs
2.5.14	CU-63: Consultar Reporte de Auditoría	1.0 hs
2.6	Sprint 6: Notificaciones Transversales y Estabilización MVP (PT-10)	25 hs
2.6.1	CU-67: Generar Notificación de Avance Técnico	1.5 hs

2.6.2	CU-68: Notificar Solicitud de Firma de Acta	1.5 hs
2.6.3	CU-69: Enviar Alerta Automática de Retraso en Obra	2.0 hs
2.6.4	CU-70: Visualizar Historial de Notificaciones de Usuario	1.5 hs
2.6.5	CU-71: Configurar Preferencia de Alerta	1.0 hs
2.6.6	CU-72: Marcar Notificación como Leída	1.5 hs
2.6.7	Pruebas de Regresión Internas, Refactor y Estabilización Transversal	16.0 hs
3.0	Fase 3: Despliegue, Transición y Cierre (Hito de Finalización)	50 hs
3.1	Pruebas integrales de estrés en escenarios reales sin conectividad (QA)	20 hs
3.2	Aprovisionamiento y despliegue de Infraestructura Cloud (Supabase)	10 hs

3.3	Compilación final y empaquetado del binario ejecutable (APK Android)	10 hs
3.4	Preparación del entorno de simulación para la Defensa y Demo en vivo	10 hs
TOTAL	ESFUERZO TOTAL PLANIFICADO PARA EL PROYECTO CONSTRUCTING	400 hs

6. COMPONENTES ADICIONALES

Para garantizar el éxito integral del ecosistema ConstructING dentro de las restricciones técnicas y académicas establecidas, se definen los siguientes planes de gestión complementarios. Debido a la naturaleza unipersonal del proyecto, estos planes se adaptan a un enfoque de auto-gestión ágil.

6.1 Plan de Seguridad (Seguridad de Datos y Criptografía)

Dada la naturaleza del sistema, cuyo valor principal radica en la auditoría inalterable, la seguridad es un componente crítico. Este plan abarca:

- **Integridad de la evidencia:** Implementación estricta de algoritmos de hash (SHA-256) sobre los reportes PDF generados y georreferenciados para garantizar su inalterabilidad lógica.
- **Gestión de Credenciales:** Las claves de acceso a las bases de datos (PostgreSQL), los tokens de la API y las credenciales de los servicios cloud (AWS/Supabase) nunca serán expuestas en el código fuente. Se gestionarán exclusivamente mediante variables de entorno local (.env).
- **Control de Accesos:** Implementación de un modelo RBAC (Role-Based Access Control) mediante NestJS para asegurar que solo los usuarios autorizados puedan generar o firmar actas.

6.2 Plan de Instalación y Despliegue (SIP)

Este componente se desarrolla de manera formal a través del entregable **Software Installation Plan (SIP)** definido en la Fase 1 del proyecto. Aborda:

- **Backend y DB:** Procedimientos paso a paso para la creación de contenedores o instancias en las capas gratuitas (Free Tiers) de AWS y Supabase.
- **Frontend:** Instrucciones para la compilación, firma y generación del archivo binario instalable (APK) para sistemas operativos Android (versión 8.0 o superior).

6.3 Plan de Entrenamiento (Capacitación Técnica) Al ser un proyecto de desarrollo individual (Greenfield), el plan de entrenamiento se traduce en un esfuerzo de auto-capacitación estructurada sin impacto en el presupuesto financiero. Se definen las siguientes áreas de formación crítica antes de iniciar los Sprints de desarrollo:

- Investigación y prueba de concepto (POC) sobre motores de bases de datos locales en Flutter (ej. SQLite, Hive o Isar) para soportar la arquitectura offline-first.

- Estudio de librerías criptográficas nativas en Dart/TypeScript para la correcta generación de firmas y hashes.

6.4 Plan de Transferencia y Mantenimiento El ciclo de vida de este proyecto académico culmina con la defensa final en diciembre de 2026. Por lo tanto, el plan de transferencia contempla:

- **Entrega del Código:** Empaquetado y transferencia del repositorio completo (GitHub) al tribunal evaluador (cátedra).
- **Mantenimiento:** El proyecto no contempla un plan de mantenimiento preventivo o evolutivo post-despliegue, ya que el alcance se congela con la entrega y aprobación del Producto Mínimo Viable (MVP) para el cierre del ciclo lectivo.

6.5 Componentes No Aplicables

Dadas las restricciones delineadas en el Propósito del Sistema (PdS) y la estructura organizacional plana, los siguientes planes se excluyen deliberadamente del alcance de este documento:

- **Plan de Gestión de Subcontratistas:** No aplica, ya que el 100% de los roles de ingeniería son asumidos por un único desarrollador.
- **Plan de Adquisición de Hardware y Facilidades:** No aplica. Se utilizará hardware personal existente (PC de escritorio y teléfono Android) y no se requieren oficinas físicas ni instalaciones especiales.

7. ÍNDICE

De acuerdo con la estructura del presente documento y para facilitar la navegación estructurada, la Tabla de Contenidos (Índice General) ha sido ubicada al comienzo del mismo. Adicionalmente, dado que este Plan de Desarrollo de Software se elabora, distribuye y evalúa exclusivamente en formato electrónico, se omite de forma deliberada la inclusión de un índice alfabético de términos en esta sección. Los lectores y el tribunal evaluador pueden utilizar la funcionalidad de búsqueda nativa (Ctrl+F / Cmd+F) de su visor de documentos para localizar términos, métricas o conceptos específicos de manera directa y eficiente.

8. APÉNDICES

El presente Plan de Desarrollo de Software no contempla apéndices al final del documento. Siguiendo el principio de separación de responsabilidades y para mantener la concisión técnica del cuerpo principal, toda la información de apoyo ha sido modularizada.

Los artefactos arquitectónicos, detalles de diseño y especificaciones complementarias (tales como la Especificación de Requerimientos de Software - ERS, Especificación de Casos de Uso, Modelo Relacional de la Base de Datos, Matrices de Riesgo y Prototipos de Interfaz de Usuario) han sido elaborados como documentos anexos independientes. Dichos documentos operan como parte integral del ecosistema documental de **ConstructING** y se encuentran debidamente referenciados en las secciones pertinentes de este plan

